
microgpt_sft.py — An Annotated Guide

LoRA Supervised Fine-Tuning in Pure Python

@nealrichter + Amazon Kiro

June 29, 2026

Abstract

This document walks through `microgpt_sft.py`, which implements LoRA (Low-Rank Adaptation) supervised fine-tuning on top of a pretrained microgpt model. It demonstrates the complete SFT pipeline: loading a frozen base model, attaching trainable low-rank adapters, training with response-only loss masking, and merging adapters back into the base weights. All in pure Python with zero dependencies.

This is Stage 2 of the teaching pipeline: **pretrain** (`microgpt.py`) → **LoRA SFT** (this file) → **DPO** (`microgpt_dpo.py`). An optional visualization module (`microgpt_viz.py`, invoked via `--viz`) renders ASCII loss sparklines and attention heat maps. An example training run is included as an appendix.

Source: https://github.com/nealrichter/research/blob/main/microgpt/microgpt_sft.py

Contents

1 Overview	1
2 Frozen Base Model	1
3 LoRA Adapters	2
4 LoRA-Augmented Forward Pass	2
5 Response-Only Loss Masking	3
6 LoRA Merge and Export	3
7 Usage	4
Appendix: Example Training Run	5

1 Overview

1. **Load:** Pretrained weights from `model.json` (frozen).
2. **Adapt:** LoRA on Q and V projections (256 trainable params).
3. **Train:** Response-only cross-entropy on instruction→response pairs.
4. **Merge:** Fold adapters into base weights → `model_sft.json`.

Parameter	Value
LoRA rank (r)	4
LoRA alpha (α)	8
Target modules	attn_wq, attn_wv
Trainable params	256 (5.7% of base)
Epochs	50

Table 1: LoRA SFT configuration

2 Frozen Base Model

```

95 state_dict = {k: [[Value(w) for w in row] for row in mat]
96                 for k, mat in model['weights'].items()}
97
98 # Extend embedding tables for SEP token if needed
99 for key in ('wte', 'lm_head'):
100     if len(state_dict[key]) < vocab_size:
101         state_dict[key].append([Value(random.gauss(0, 0.08))
102                                 for _ in range(n_embd)])
103
104 block_size = 32
105 while len(state_dict['wpe']) < block_size:
106     state_dict['wpe'].append([Value(random.gauss(0, 0.08))
107                               for _ in range(n_embd)])
108
109 base_params = [p for mat in state_dict.values()
110                for row in mat for p in row]

```

3 LoRA Adapters

LoRA [1] adds trainable low-rank matrices while keeping base weights frozen:

$$h = W_0 x + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}} \cdot x$$

W_{down} is randomly initialized; W_{up} is zeros (no perturbation at init).

```

110 # --- LoRA adapters on Q and V projections ---
111 lora_rank = 4
112 lora_alpha = 8
113 lora_scale = lora_alpha / lora_rank
114
115 matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std))
116     for _ in range(nin)] for _ in range(nout)]
117
118 lora_dict = {}
119 for i in range(n_layer):
120     lora_dict[f'layer{i}.attn_wq.down'] = matrix(lora_rank, n_embd)
121     lora_dict[f'layer{i}.attn_wq.up'] = [[Value(0.0)
122     for _ in range(lora_rank)] for _ in range(n_embd)]
123     lora_dict[f'layer{i}.attn_wv.down'] = matrix(lora_rank, n_embd)
124     lora_dict[f'layer{i}.attn_wv.up'] = [[Value(0.0)
125     for _ in range(lora_rank)] for _ in range(n_embd)]
126
127 lora_params = [p for mat in lora_dict.values()

```

```
128     for row in mat for p in row]
```

4 LoRA-Augmented Forward Pass

```
128 def linear(x, w):
129     return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in w]
130
131 def linear_lora(x, w_base, w_down, w_up):
132     """h = W_base @ x + scale * W_up @ (W_down @ x)"""
133     return [b + lora_scale * l for b, l in zip(
134         linear(x, w_base), linear(linear(x, w_down), w_up))]
```

Applied only to Q and V in attention (K and MLP unchanged) [1]:

```
144 def gpt(token_id, pos_id, keys, values):
145     ...
146     q = linear_lora(x, state_dict[f'layer{li}.attn_wq'],
147         lora_dict[f'layer{li}.attn_wq.down'],
148         lora_dict[f'layer{li}.attn_wq.up'])
149     k = linear(x, state_dict[f'layer{li}.attn_wk'])
150     v = linear_lora(x, state_dict[f'layer{li}.attn_wv'],
151         lora_dict[f'layer{li}.attn_wv.down'],
152         lora_dict[f'layer{li}.attn_wv.up'])
```

5 Response-Only Loss Masking

Loss computed only on response tokens [2]:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{|R|} \sum_{j \in R} \log P(t_j \mid t_1, \dots, t_{j-1})$$

where R = positions after SEP.

```
224     for instruction, response in sft_data:
225         instr_tok = encode(instruction)
226         resp_tok = encode(response)
227         tokens = [BOS] + instr_tok + [SEP] + resp_tok + [BOS]
228         n = min(block_size, len(tokens) - 1)
229         resp_start = len(instr_tok) + 2
230
231         keys, values = [[] for _ in range(n_layer)],
232             [[] for _ in range(n_layer)]
233         losses = []
234         for pos_id in range(n):
235             logits = gpt(tokens[pos_id], pos_id, keys, values)
236             if pos_id >= resp_start - 1:
237                 probs = softmax(logits)
238                 losses.append(-probs[tokens[pos_id + 1]].log())
239
240         loss = (1 / len(losses)) * sum(losses)
```

6 LoRA Merge and Export

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}}$$

```
268 # --- Merge LoRA into base and save ---
269 for i in range(n_layer):
270     for key in (f'layer{i}.attn_wq', f'layer{i}.attn_wv'):
271         w_base = state_dict[key]
272         w_down = lora_dict[f'{key}.down']
273         w_up = lora_dict[f'{key}.up']
274         for r in range(len(w_base)):
275             for c in range(len(w_base[0])):
276                 w_base[r][c].data += lora_scale * sum(
277                     w_up[r][k].data * w_down[k][c].data
278                     for k in range(lora_rank))
```

7 Usage

```
1 python3 microgpt.py           # pretrain (required first)
2 python3 microgpt_sft.py       # LoRA SFT -> model_sft.json
3 python3 microgpt_sft.py -i    # inference from model_sft.json
4 python3 microgpt_sft.py --viz 50 # train with visualization
5 python3 microgpt_sft.py -h    # help
```

References

- [1] Hu, J.E., et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [2] Wei, J., et al. *Finetuned Language Models Are Zero-Shot Learners*. ICLR, 2022. <https://arxiv.org/abs/2109.01652>
- [3] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. ICLR, 2015. <https://arxiv.org/abs/1412.6980>

Appendix: Example Training Run

TBA